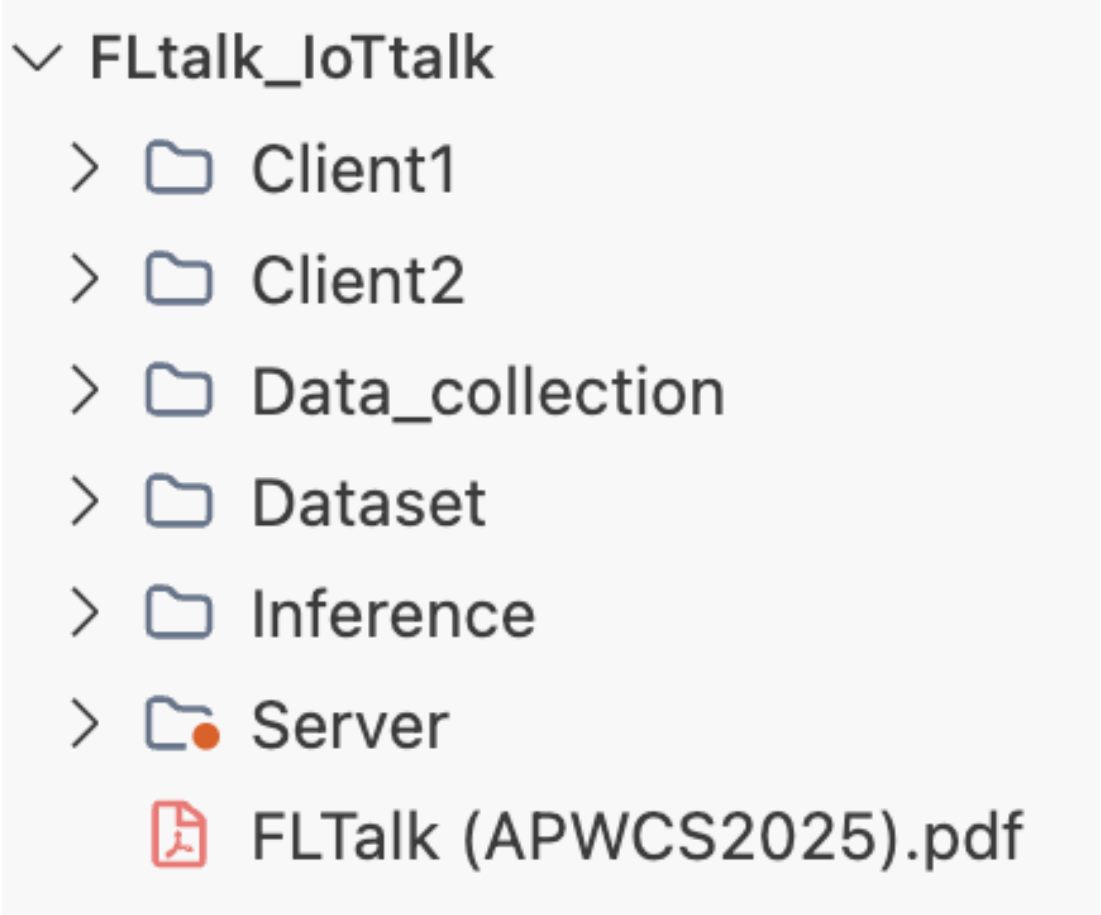
FLtalk v0

2026/10/20

Project Structure



這份專案是基於發表於 **IEEE APWCS 2025** 的學術論文 **《Implementing Federated Learning via IoTtalk: A Smart Home Case Study》**（由國立臺灣海洋大學資工系團隊發表，論文完整內容即為根目錄下的 `FLTalk (APWCS2025).pdf`）所開發的完整實作系統。

以下為本專案的核心目標、系統架構、各階段運作流程及專案程式碼結構的詳細剖析：

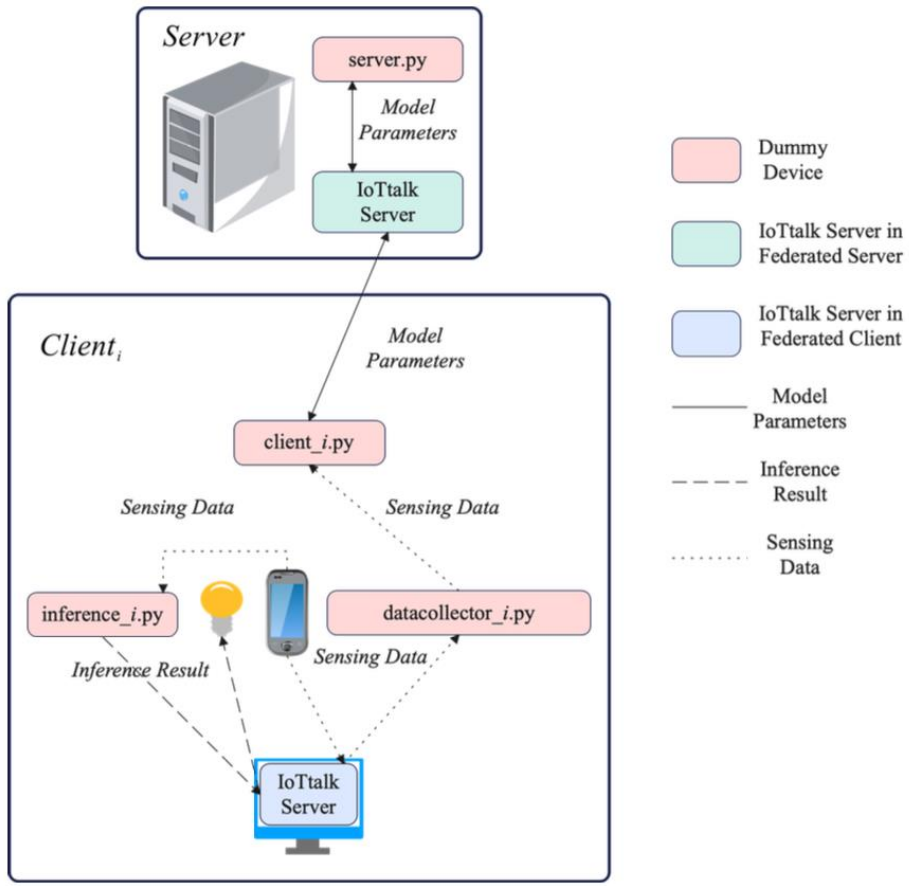
一、 專案核心目標與背景

1. **解決智慧家庭 AIoT 的隱私痛點**： 在智慧家庭環境中，感測器數據（如加速度計、陀螺儀等）高度涉及使用者的行為模式與日常生活隱私。傳統中央集權式 AI 需要將原始感測資料全數上傳伺服器，存在嚴重的資料外洩風險。
2. **導入聯邦學習（Federated Learning, FL）**： 透過聯邦學習機制，用戶的原始數據僅保留在本地端，僅上傳**模型參數（Weights & Intercept）**到中央伺服器進行聚合（Aggregation），訓練出全域模型（Global Model）。
3. **結合 IoTtalk 實現低代碼（Low-Code/No-Code, LCNC）快速部署**： IoTtalk 原生以圖形化拖拉介面和 **Dummy Device（虛擬裝置）** 機制見長，但原生不支援 FL。本專案提出 **FLtalk** 框架，利用 IoTtalk 的 Device Feature (`Dummy_Sensor`、`Dummy_Control`) 與 MQTT 通訊，以極低的程式碼負擔快速構建出適用於邊緣 IoT 設備的**單輪聯邦學習（One-Shot Federated Learning, OFL）**系統。

二、 系統架構設計

系統分為兩個層級的 IoTtalk 部署（如論文圖 1 所示）：

1. **Federated Server（聯邦中央伺服器）**：
    - 運行一個專供 FL 參數匯聚的中央 IoTtalk Server。
    - 部署 Dummy Device `Server/SA.py`，透過 `Dummy_Control` 接收來自各 Client 的模型參數，計算平均值後產生全域模型，再透過 `Dummy_Sensor` 廣播回發給各 Client。
2. **Federated Client（聯邦客戶端，本專案實作 Client 1 與 Client 2）**：
    - 每個 Client 在本地運行**獨立的 IoTtalk Server**（確保原始感測資料不會傳到外部網絡）。
    - 每個 Client 內包含三個 Dummy Device：
        - **數據收集裝置**：`Data_collection/data_collector.py`（連接本地 IoTtalk，收集手機感測器資訊）。
        - **聯邦通訊裝置**：`Client1/SA.py` / `Client2/SA.py`（連線至中央伺服器 IoTtalk，負責上傳本地模型參數並取回全域模型）。
        - **即時推論裝置**：`Inference/SA.py`（只連本地 IoTtalk，載入全域模型進行手勢識別，並控制虛擬燈泡）。



三、 三大運作階段（Workflow）

mermaid

```
flowchart LR
    subgraph Phase1["Phase 1: 數據收集與前處理"]
        A[智慧型手機 3軸加速度計 + 3軸陀螺儀] -->|每 0.1s 取樣| B[IoTtalk Join 合併為
6D]
        B --> C[每 10 筆合併為 60D 動作窗口]
        C --> D[標註: 畫圈=ON / 交叉=OFF]
    end

    subgraph Phase2["Phase 2: 本地訓練與單輪聯邦彙整"]
        D -->|切分資料| E1[Client 1 本地訓練 LR]
        D -->|切分資料| E2[Client 2 本地訓練 LR]
        E1 -->|上傳參數 JSON| F[Server: FedAvg 參數平均]
        E2 -->|上傳參數 JSON| F
        F -->|分發全域模型| G1[Client 1 儲存全域模型]
        F -->|分發全域模型| G2[Client 2 儲存全域模型]
    end

    subgraph Phase3["Phase 3: 邊緣即時推論"]
        H[即時手勢感測串流] --> I[Inference.py 本地推論]
        G1 -.-> I
        I -->|P >= 0.5| J1[開啟虛擬燈泡 ON / 100]
        I -->|P < 0.5| J2[關閉虛擬燈泡 OFF / 0]
    end
```

Data Collection Phase

- **感測訊號**：手機內建加速度計（acc_x, acc_y, acc_z）與陀螺儀（$gyro_x, gyro_y, gyro_z$），取樣頻率為每 0.1 秒 1 筆。
- **特徵工程**：每個手勢時長約 1 秒，因此將連續 10 筆取樣（$10 \times 6 = 60$ 維特徵）整合成一個樣本。
- **手勢定義**：
    - 手機畫圈（Circular motion）$\rightarrow$ 開啟家電「ON」（標籤 `1`，`circle.csv`）
    - 手機上下/十字擺動（Crosswise motion）$\rightarrow$ 關閉家電「OFF」（標籤 `0`，`cross.csv`）
- 共收集 1200 筆樣本，經 `data_preprocess.py` 分配：
    - 1000 筆為訓練集（Client 1、Client 2 各自分得 500 筆）。
    - 200 筆獨立作為測試集（`test_data.csv`）。

Training Phase

- **模型選用**：邏輯回歸（Logistic Regression, LR），計算量小且非常適合邊緣 IoT 硬體。
- **本地訓練**：
    - `Client1/local_training.py` 訓練後將權重與截距匯出為 `params_client1.json`。
    - Client 2 同樣產生 `params_client2.json`。

Aggregation Phase

- **單輪聚合（One-Shot FedAvg）**：
  - 各 Client 透過 MQTT 將參數傳送給 Server。
  - Server 在 `Server/SA.py` 中計算算術平均： $$W_{\text{global}} = \frac{W_1 + W_2}{2}, \quad b_{\text{global}} = \frac{b_1 + b_2}{2}$$
  - 將產生之 `global_model.json` 廣播發回 Client。
  - 透過單輪通訊解決 IoT 設備頻寬與連線不穩定的問題。

Inference Phase

- 在 `Inference/SA.py` 載入收到之全域模型。
- 本地感測器持續串流資料，每湊滿 10 個時步（60 維向量）即透過 Sigmoid 函數計算分類機率： $$P(\text{ON}) = \frac{1}{1 + e^{-(W_{\text{global}} \cdot X + b_{\text{global}})}}$$
- 機率 ≥ 0.5 判定為開燈（設定燈泡亮度數值為 100），< 0.5 判定為關燈（數值為 0），並將控制訊號送達虛擬燈泡。

專案架構

```
FLtalk_IoTtalk/
├── FLTalk (APWCS2025).pdf          # 研討會論文原文
├── Dataset/                        # 原始與切分後的感測資料集
│   ├── circle.csv / cross.csv      # 開/關燈動作的原始感測數據
│   ├── sensor_data.csv             # 合併後貼好標籤的完整資料集
│   ├── client1_train.csv           # 分配給 Client 1 的訓練資料 (500筆)
│   ├── client2_train.csv           # 分配給 Client 2 的訓練資料 (500筆)
│   └── test_data.csv               # 獨立評估測試集 (200筆)
├── Data_collection/                # 資料收集與前處理工具
│   ├── SA.py & data_collector.py   # IoTtalk 資料收集 Dummy Device
│   ├── data_process.py             # 轉換為 60D 動作窗口與標籤
│   └── data_preprocess.py          # 切割訓練/測試資料與分散式劃分
├── Client1/ & Client2/             # 聯邦學習客戶端
│   ├── local_training.py           # 本地模型訓練、繪製混淆矩陣/ROC，並輸出參數 JSON
│   ├── SA.py                       # Client 端 IoTtalk 通訊介面（上傳本地參數 / 接收
全域模型）
│   └── global_model_received.json  # 收到並儲存的全域模型
├── Server/                         # 聯邦學習中央伺服器
│   ├── SA.py                       # 接收各 Client 參數、執行平均聚合並發布全域模型
│   └── model_eval.py               # 在測試集上評估全域模型效能指標 (Accuracy, ROC,
PR Curve)
└── Inference/                      # 本地推論端
    └── SA.py                       # 載入全域模型，即時接收感測數據並進行推論控制虛擬燈泡
```